

PROYECTO FIN DE CICLO FORMATIVO



IES FLORIDABLANCA
DEPARTAMENTO DE INFORMÁTICA

TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

Aplicación web de préstamo de libros

AUTOR(ES)

D. Sergio Fernández Muñoz
D. Abdennour Mekhissi Berakeche
D. Erik Conesa Belando

TUTOR

D. Juan Antonio Cuello Alarcón

MURCIA, 20 DE MARZO DE 2026



Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License

ÍNDICE GENERAL

Contenido

1. Introducción	5
1.1 Objetivo del proyecto	5
1.2 Despliegue de la aplicación	5
1.2.1 Guía de puesta en marcha.....	6
2. Requisitos del proyecto	7
2.1 Requisitos funcionales.....	7
2.2 Mejoras adicionales.....	8
3. Diseño de la aplicación.....	9
3.1 Arquitectura general del proyecto.....	9
3.2 Diseño de la base de datos	10
3.2.1 Modelo de datos	10
3.2.2 Diseño relacional	13
3.3 Diseño de la interfaz de usuario	14
3.4 Descripción de casos de uso y pruebas	17
4. Implementación	25
4.1 Tecnologías utilizadas	25
4.2 Documentación de la API	25
4.2.1 URL Base.....	25
4.2.2 Autenticación	25
4.2.3 Endpoints.....	26
4.2.4 Códigos de Respuesta HTTP	35
4.2.5 Ejemplos de uso	36
4.2.6 Resumen de Endpoints.....	38
5. Bibliografía y recursos usados.....	39
6. Anexos.....	40

1. Introducción

1.1 Objetivo del proyecto

El objetivo de este proyecto es desarrollar una aplicación web que permita gestionar el préstamo de libros entre usuarios registrados. La aplicación permitirá a los usuarios registrarse en el sistema, añadir libros de su propiedad y solicitar el préstamo de libros disponibles publicados por otros usuarios.

El sistema gestionará la información relacionada con los usuarios, los libros y los préstamos, permitiendo mantener un control del estado de cada libro y del tiempo que lleva prestado. Además, los usuarios podrán consultar los libros que han prestado y los que han solicitado, así como utilizar un buscador para encontrar libros según distintos criterios como el título, autor o género.

Asimismo, la aplicación incluirá una interfaz de administración que permitirá supervisar los préstamos realizados y gestionar los usuarios del sistema. El desarrollo de este proyecto permite aplicar los conocimientos adquiridos en el ciclo formativo relacionados con el desarrollo de aplicaciones web, el uso de bases de datos, el diseño de interfaces y la implementación de sistemas de gestión de información.

Este proyecto tiene como finalidad proporcionar una solución funcional, intuitiva y accesible desde distintos dispositivos, cumpliendo los requisitos establecidos y aplicando buenas prácticas en el desarrollo de aplicaciones web.

1.2 Despliegue de la aplicación

La aplicación ha sido desplegada en un servidor VPS, donde se alojan tanto el frontend como el backend. El frontend ha sido desarrollado con Angular, mientras que el backend consiste en una API REST desarrollada con Laravel, encargada de gestionar la lógica de negocio y el acceso a los datos.

El servidor utiliza Apache como servidor web y MariaDB como sistema de gestión de base de datos, alojado en el mismo VPS. La aplicación es accesible a través de navegador web y permite la conexión simultánea de múltiples usuarios, garantizando el correcto funcionamiento del sistema.

En el siguiente apartado encontrarás información de como desplegar tú mismo el proyecto en tu servidor.

1.2.1 Guía de puesta en marcha

TODO

2. Requisitos del proyecto

En este apartado se recogen los requisitos que debe cumplir la aplicación para considerarse funcional y completa, definidos a partir del análisis de las necesidades de los usuarios y los objetivos del proyecto. Se distingue entre los requisitos funcionales, que describen las funcionalidades que el sistema debe ofrecer tanto a usuarios como a administradores, y una serie de mejoras adicionales que amplían las capacidades básicas de la aplicación.

2.1 Requisitos funcionales

- RF-01: Registro de usuarios mediante nombre, apellidos, email y contraseña.
- RF-02: Inicio de sesión de usuarios mediante email y contraseña.
- RF-03: Cierre de sesión e invalidación del token de acceso.
- RF-04: Posibilidad de autenticación mediante cuenta de Google.
- RF-05: Consulta del perfil propio por parte del usuario.
- RF-06: Actualización del perfil propio por parte del usuario.
- RF-07: Listado de todos los usuarios registrados (administradores).
- RF-08: Administradores pueden dar de baja usuarios sin préstamos activos.
- RF-09: Alta de libros por parte de los usuarios, indicando título, autor, género y año.
- RF-10: Visualización de los libros disponibles en la plataforma.
- RF-11: Buscador de libros por título, autor o género.
- RF-12: Consulta del detalle de un libro concreto.
- RF-13: Edición de libros por parte de su propietario.
- RF-14: Eliminación de libros por parte de su propietario.
- RF-15: Capacidad de dar de alta libros mediante archivo CSV.
- RF-16: Listado de copias de un libro con su estado.
- RF-17: Alta de nuevas copias de un libro por parte del propietario.
- RF-18: Eliminación de copias sin préstamos activos.
- RF-19 Solicitud de préstamo de libros entre usuarios.
- RF-20 Devolución de libros prestados.
- RF-21: Consulta de los préstamos activos propios del usuario.
- RF-22: Visualización de los libros prestados como propietario.
- RF-23: Gestión de todos los préstamos del sistema (administradores).
- RF-24: Gestión del catálogo de géneros.
- RF-25: Gestión del catálogo de autores.
- RF-26: Almacenamiento persistente de datos.
- RF-27: API REST desarrollada para la gestión de los datos.
- RF-28: Interfaz web responsive sencilla para cualquier usuario.

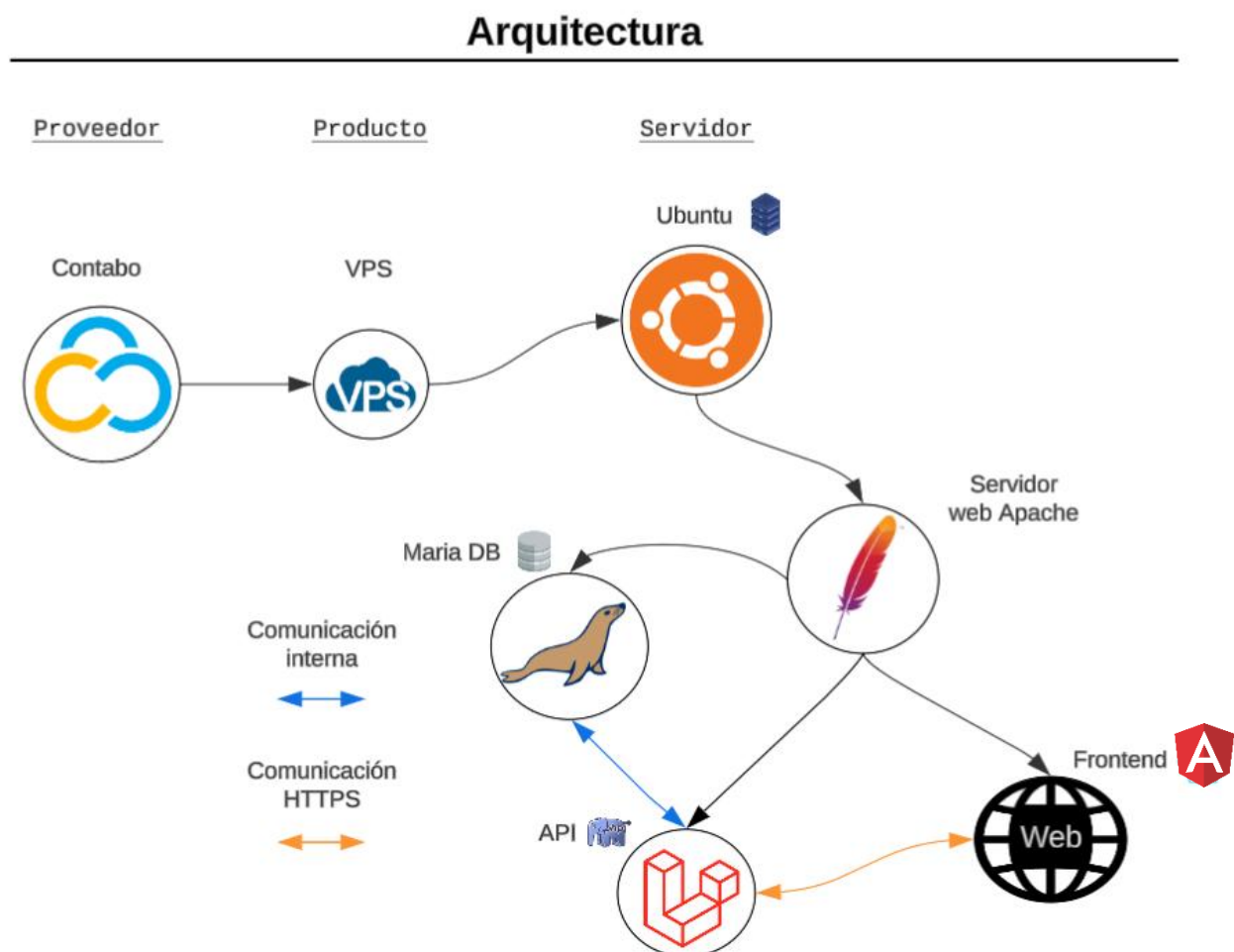
2.2 Mejoras adicionales

- Permitir la subida de imágenes de portada de los libros.
- Implementar sistema de valoraciones de libros.

3. Diseño de la aplicación

3.1 Arquitectura general del proyecto

La arquitectura del proyecto comienza con la contratación de un servicio VPS en la plataforma Contabo, donde se alquila un servidor con sistema operativo Ubuntu. En este servidor se aloja una base de datos MariaDB, una API desarrollada en PHP utilizando el framework Laravel y un frontend desarrollado con Angular. Todos estos servicios son gestionados por el servidor web Apache. La conexión al servidor se realiza mediante el protocolo HTTPS, lo que garantiza una comunicación segura y protege la información transmitida entre el cliente y el servidor.



3.2 Diseño de la base de datos

A continuación, tienes el modelo de datos de todas las tablas y una descripción de sus campos. Por otro lado, también tienes una imagen que representa el diseño relacional de todas las tablas.

3.2.1 Modelo de datos

- **Usuarios (users)**

Campo	Tipo	Descripción
id	integer	Identificador único autoincremental
name	string(45)	Nombre del usuario
lastname	string(45)	Apellidos del usuario
email	string(100)	Email único, usado para login
password	string(255)	Contraseña cifrada (bcrypt)
role	enum	Rol del usuario: 'user' o 'admin'
created_at	timestamp	Fecha de registro
updated_at	timestamp	Fecha de última actualización

- **Autores (authors)**

Campo	Tipo	Descripción
id	integer	Identificador único autoincremental
name	string(45)	Nombre del autor (único)
created_at	timestamp	Fecha de creación
updated_at	timestamp	Fecha de última actualización

- **Géneros (genres)**

Campo	Tipo	Descripción
id	integer	Identificador único
name	string(45)	Nombre del género (único)
created_at	timestamp	Fecha de creación
updated_at	timestamp	Fecha de última actualización

- Libros (books)

Campo	Tipo	Descripción
id	integer	Identificador único
title	string(45)	Título del libro
genre_id	integer (FK)	Referencia al género
author_id	integer (FK)	Referencia al autor
owner_id	integer (FK)	Usuario propietario del libro
cover_image	string(255)	Ruta o URL de la portada
publication_year	year(4)	Año de publicación
created_at	timestamp	Fecha de alta del libro
updated_at	timestamp	Fecha de última actualización

- Copias (copies)

Campo	Tipo	Descripción
id	integer	Identificador único
book_id	integer (FK)	Libro al que pertenece la copia
code	string(45)	Código único de la copia
loan_date	date	Fecha en que se realiza el préstamo
state	enum('available','borrowed')	Estado de la copia
created_at	timestamp	Fecha de creación
updated_at	timestamp	Fecha de última actualización

- Préstamos (loans)

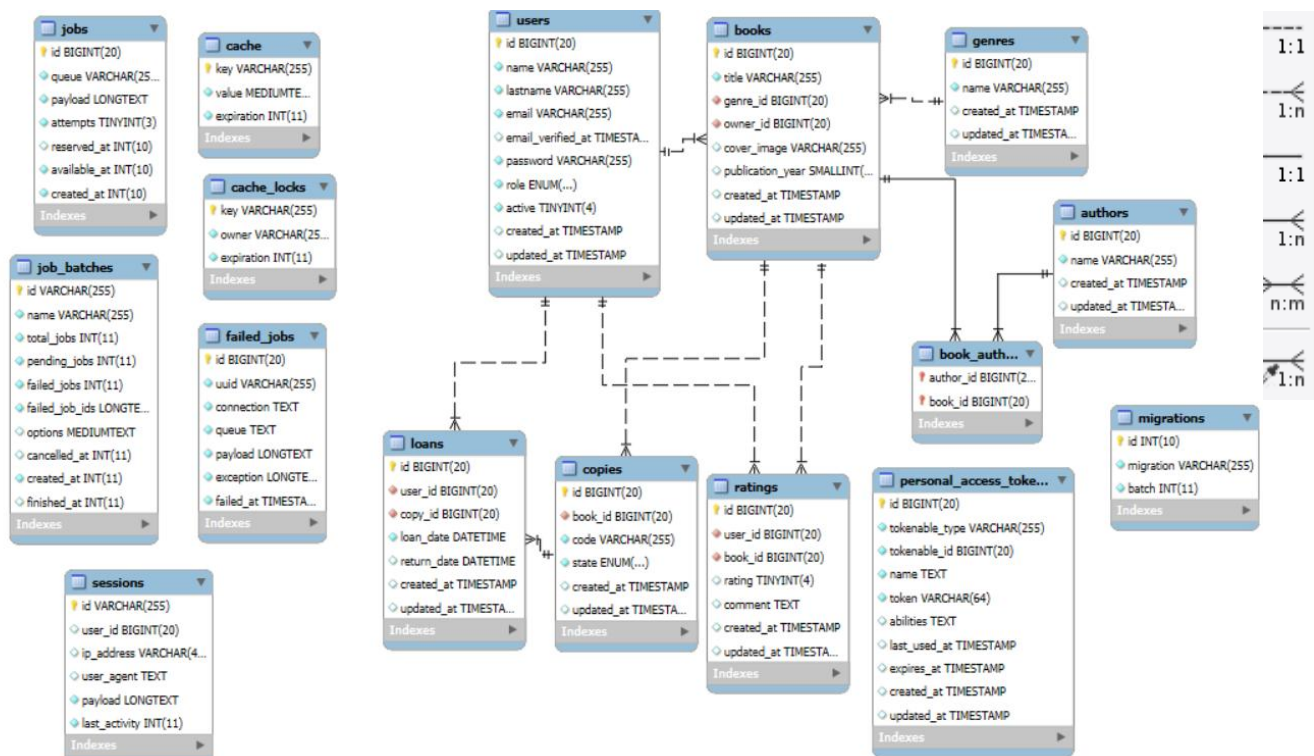
Campo	Tipo	Descripción
id	integer	Identificador único
user_id	integer (FK)	Usuario que realiza el préstamo
copy_id	integer (FK)	Copia del libro prestada
loan_date	date	Fecha en que se realiza el préstamo
return_date	date	Fecha de devolución (null si activo)
created_at	timestamp	Fecha de creación
updated_at	timestamp	Fecha de última actualización

- **Valoraciones (loans)**

Campo	Tipo	Descripción
id	integer	Identificador único
user_id	integer (FK)	Usuario que valora
book_id	integer (FK)	Libro valorado
rating	tinyint(1)	Puntuación del libro
comment	text	Comentario del usuario
created_at	timestamp	Fecha de creación del registro
updated_at	timestamp	Fecha de última actualización

3.2.2 Diseño relacional

- Amarillo: PK
- Rojo: FK.
- Llave roja: PK y FK.
- Azul claro: *Not null*.
- Rombo azul vacío: Sin restricciones.
- Rombo rojo vacío: FK, único y *null*.



3.3 Diseño de la interfaz de usuario

Estas son unas imágenes de los bocetos en los que nos hemos fijado para diseñar todas las vistas del frontend. Para el desarrollo de estos bocetos hemos utilizado Figma

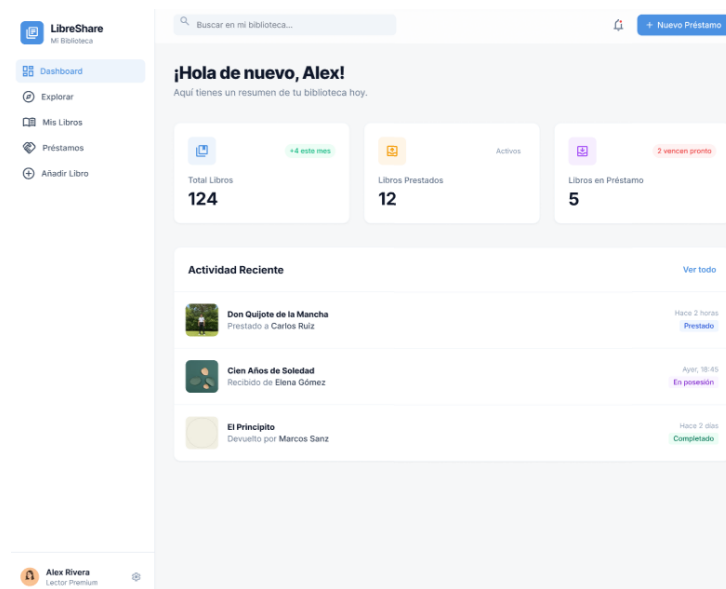
- Login

The login page features the LibreShare logo at the top left. The main content is a white card with a blue circular icon containing a right arrow. Below this is the heading "Welcome Back" and the subtext "Log in to your LibreShare account". The form includes an "Email Address" field with the placeholder "name@example.com", a "Password" field with masked characters and a "Forgot password?" link, and a "Keep me logged in" checkbox. A prominent blue "Sign In" button with a right arrow is positioned below the form. Underneath, it says "OR CONTINUE WITH" followed by "Google" and "GitHub" buttons. At the bottom of the card, it asks "New to LibreShare?" and provides a "Create Account" link. The footer contains links for "Privacy Policy", "Terms of Service", and "Help Center", along with a copyright notice: "© 2024 LibreShare Inc. All rights reserved."

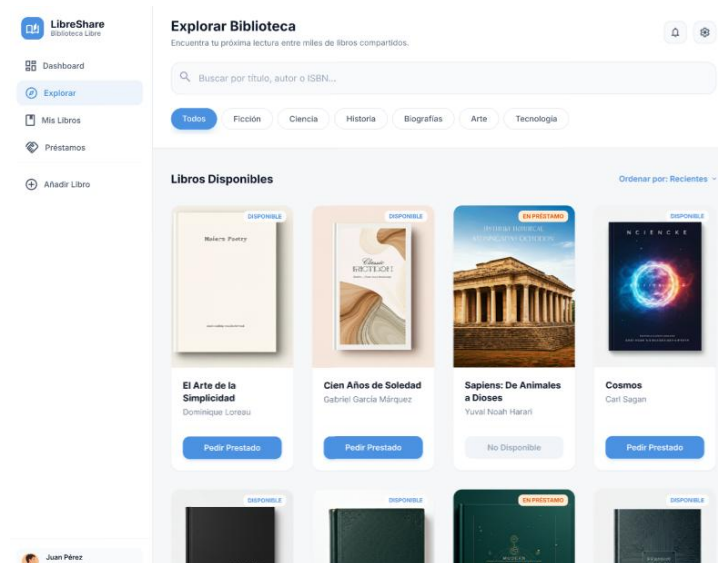
- Registro

The registration page features the LibreShare logo at the top center with the tagline "Join our open community". The main content is a white card with the heading "Create Account". The form includes "First Name" (placeholder "Jane") and "Last Name" (placeholder "Doe") fields, an "Email Address" field with the placeholder "jane@example.com", a "Password" field with masked characters and a strength indicator, and a "Confirm Password" field with masked characters. A prominent blue "Create Account" button is positioned below the form. At the bottom of the card, it asks "Already have an account?" and provides a "Login" link. The footer contains links for "Privacy Policy", "Terms of Service", and "Support".

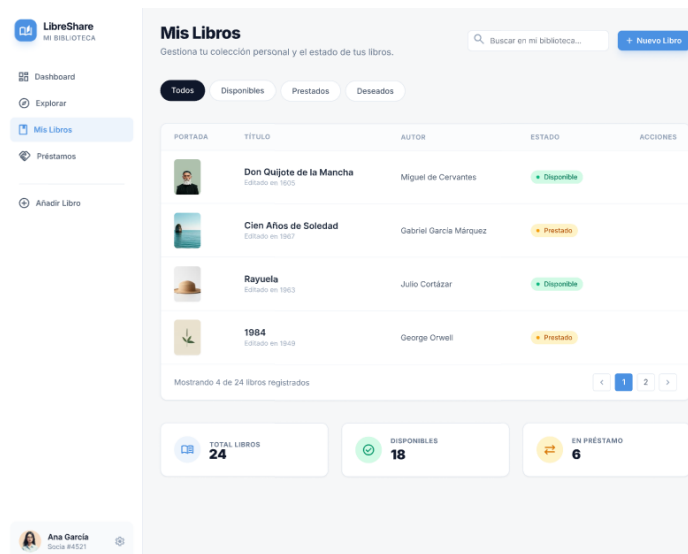
- Dashboard



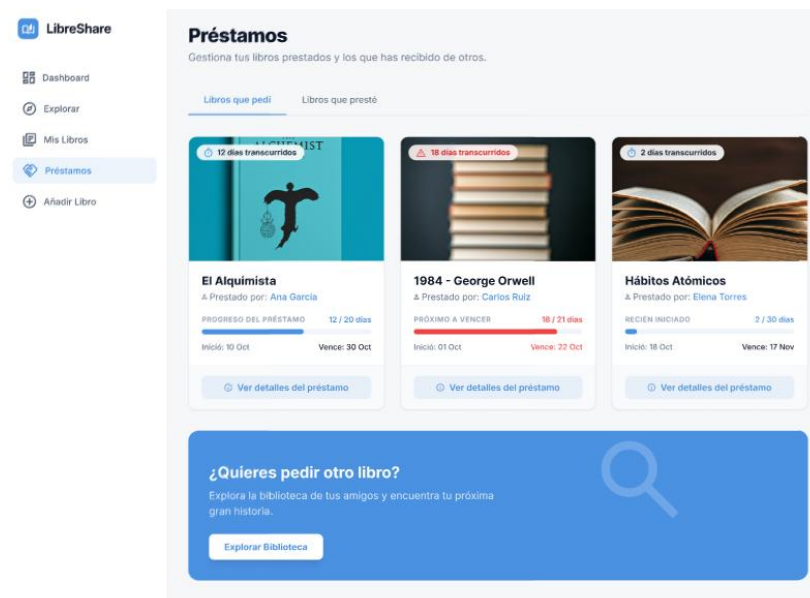
- Explorar



- Mis libros



- Préstamos



3.4 Descripción de casos de uso y pruebas

En este apartado se van a describir todas y cada de las funcionalidades que cumple la aplicación de manera exitosa, habiendo obtenido resultados positivos después de realizar pruebas a cada una de ellas. Los requisitos 26, 27 y 28 no se describen porque son requisitos más técnicos que se aplican al proyecto y no a los usuarios.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-01
Nombre	Registro de usuario
Actores	Usuario no registrado
Descripción	Un usuario no registrado podrá crear una nueva cuenta en el sistema proporcionando su nombre, apellidos, dirección de correo electrónico y una contraseña. Al completar el registro, recibirá un token de acceso y quedará autenticado en el sistema.
Precondición	El usuario no debe tener una cuenta registrada con el mismo correo electrónico.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-02
Nombre	Inicio de sesión
Actores	Todos los usuarios registrados
Descripción	Los usuarios registrados podrán autenticarse en la aplicación introduciendo su correo electrónico y contraseña. Tras una autenticación exitosa, el sistema devolverá un token de acceso que el cliente deberá incluir en las peticiones posteriores.
Precondición	El usuario debe existir en el sistema y sus credenciales deben ser correctas.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-03
Nombre	Cierre de sesión
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán cerrar su sesión activa. El sistema invalidará el token de acceso actual, de forma que no pueda ser reutilizado en peticiones posteriores.
Precondición	El usuario debe haber iniciado sesión previamente y disponer de un token válido.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-04
Nombre	Inicio de sesión con Google
Actores	Todos los usuarios
Descripción	Los usuarios podrán autenticarse en la aplicación utilizando su cuenta de Google mediante el protocolo OAuth2. El sistema verificará el token de Google proporcionado y, si es válido, creará o recuperará la cuenta del usuario y devolverá un token de acceso propio.
Precondición	El usuario debe disponer de una cuenta de Google activa y haber concedido los permisos necesarios en Google.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-05
Nombre	Consulta del perfil propio
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán consultar los datos de su propio perfil, incluyendo nombre, apellidos, correo electrónico y rol en el sistema. La contraseña no será incluida en la respuesta.
Precondición	El usuario debe haber iniciado sesión previamente.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-06
Nombre	Actualización del perfil propio
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán modificar sus propios datos personales, como nombre, apellidos, correo electrónico y contraseña. Solo se actualizarán los campos que el usuario proporcione en la solicitud.
Precondición	El usuario debe haber iniciado sesión previamente. En caso de cambio de correo, el nuevo email no debe estar registrado por otro usuario.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-07
Nombre	Listado de todos los usuarios
Actores	Administradores
Descripción	Los administradores podrán obtener un listado paginado de todos los usuarios registrados en el sistema, con todos sus datos excepto la contraseña.
Precondición	El usuario debe haber iniciado sesión con rol de administrador.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-08
Nombre	Eliminación de usuario
Actores	Administradores
Descripción	Los administradores podrán eliminar un usuario del sistema siempre que este no tenga préstamos activos asociados. Si el usuario tiene préstamos activos, la operación será rechazada.
Precondición	El usuario debe haber iniciado sesión con rol de administrador. El usuario a eliminar debe existir en el sistema y no tener préstamos activos.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-09
Nombre	Alta de libro
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán dar de alta un nuevo libro en el sistema indicando el título, el autor, el género y opcionalmente el año de publicación. El propietario del libro se asignará automáticamente al usuario que realiza la solicitud.
Precondición	El usuario debe haber iniciado sesión previamente. El autor y el género indicados deben existir en el sistema.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-10
Nombre	Visualización del catálogo de libros
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán consultar el catálogo completo de libros disponibles en la plataforma. La respuesta incluirá información del autor, género, propietario y número de copias disponibles de cada libro. Los resultados se presentarán de forma paginada.
Precondición	El usuario debe haber iniciado sesión previamente.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-11
Nombre	Búsqueda de libros
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán buscar libros dentro del catálogo utilizando filtros por título, autor o género. La búsqueda puede combinarse con paginación para gestionar grandes volúmenes de resultados.
Precondición	El usuario debe haber iniciado sesión previamente.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-12
Nombre	Consulta de un libro concreto
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán consultar los detalles completos de un libro específico mediante su identificador, incluyendo autor, género, propietario y el listado de copias disponibles.
Precondición	El usuario debe haber iniciado sesión previamente. El libro debe existir en el sistema.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-13
Nombre	Edición de libro
Actores	Usuario propietario del libro
Descripción	El propietario de un libro podrá modificar sus datos, como el título, autor, género o año de publicación. Solo el usuario propietario tendrá permiso para editar el libro.
Precondición	El usuario debe haber iniciado sesión previamente y ser el propietario del libro. El libro debe existir en el sistema.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-14
Nombre	Eliminación de libro
Actores	Usuario propietario del libro
Descripción	El propietario de un libro podrá eliminar dicho libro del sistema, siempre que ninguna de sus copias tenga préstamos activos. Si hay préstamos activos, la operación será rechazada.
Precondición	El usuario debe haber iniciado sesión previamente y ser el propietario del libro. El libro no debe tener copias con préstamos activos.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-15
Nombre	Importación masiva de libros por CSV
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán importar múltiples libros de forma masiva a través de un archivo CSV con las columnas: título, autor, género y año de publicación. El sistema creará automáticamente los autores y géneros que no existan. Los errores por fila no detendrán la importación completa.
Precondición	El usuario debe haber iniciado sesión previamente. El archivo debe estar en formato CSV con las cabeceras correctas.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-16
Nombre	Listado de copias de un libro
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán consultar todas las copias existentes de un libro determinado, junto con su estado (disponible o prestada) y su código identificador.
Precondición	El usuario debe haber iniciado sesión previamente. El libro debe existir en el sistema.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-17
Nombre	Alta de copia de un libro
Actores	Usuario propietario del libro
Descripción	El propietario de un libro podrá añadir nuevas copias físicas del mismo, indicando un código único para cada ejemplar. La copia quedará registrada con el estado 'disponible'.
Precondición	El usuario debe haber iniciado sesión previamente y ser el propietario del libro. El código de la copia no debe estar registrado previamente.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-18
Nombre	Eliminación de copia
Actores	Usuario propietario del libro
Descripción	El propietario de un libro podrá eliminar una copia del mismo siempre que no esté prestada en ese momento. Si la copia tiene un préstamo activo, la operación será rechazada.
Precondición	El usuario debe haber iniciado sesión previamente y ser el propietario de la copia. La copia no debe estar prestada actualmente.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-19
Nombre	Solicitud de préstamo
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán solicitar en préstamo una copia disponible de un libro. El sistema registrará automáticamente la fecha de inicio del préstamo y cambiará el estado de la copia a 'prestada'.
Precondición	El usuario debe haber iniciado sesión previamente. La copia debe existir y estar en estado 'disponible'. El usuario no puede tener esa misma copia ya en préstamo.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-20
Nombre	Devolución de libro
Actores	Usuario que tiene el préstamo activo
Descripción	El usuario que tiene un préstamo activo podrá registrar la devolución del libro. El sistema registrará la fecha de devolución y cambiará el estado de la copia a 'disponible'.
Precondición	El usuario debe haber iniciado sesión previamente y ser el usuario que solicitó el préstamo. El préstamo debe estar activo (no devuelto previamente).

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-21
Nombre	Consulta de préstamos activos propios
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán consultar todos los préstamos que tienen activos actualmente, es decir, los libros que han solicitado y aún no han devuelto. La respuesta incluirá los días transcurridos desde el inicio de cada préstamo.
Precondición	El usuario debe haber iniciado sesión previamente.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-22
Nombre	Consulta de libros prestados como propietario
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán consultar qué copias de sus propios libros están siendo prestadas en ese momento a otros usuarios, incluyendo información del prestatario y los días transcurridos desde el inicio del préstamo.
Precondición	El usuario debe haber iniciado sesión previamente.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-23
Nombre	Gestión de préstamos (administrador)
Actores	Administradores
Descripción	Los administradores podrán consultar todos los préstamos del sistema, tanto activos como históricos, con posibilidad de filtrado por usuario o por estado (activo/cerrado). Los resultados se presentarán de forma paginada.
Precondición	El usuario debe haber iniciado sesión con rol de administrador.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-24
Nombre	Gestión de géneros
Actores	Administradores / Usuarios registrados
Descripción	Los usuarios autenticados podrán consultar el listado de géneros disponibles. Los administradores, además, podrán crear nuevos géneros. No se admitirán géneros duplicados.
Precondición	Para consultar géneros: el usuario debe haber iniciado sesión. Para crear géneros: el usuario debe tener rol de administrador.

DESCRIPCIÓN DE CASO DE USO	
Requisito Funcional	RF-25
Nombre	Gestión de autores
Actores	Todos los usuarios registrados
Descripción	Los usuarios autenticados podrán consultar el listado de autores disponibles y buscar por nombre. También podrán crear nuevos autores indicando su nombre, siempre que no exista previamente en el sistema.
Precondición	El usuario debe haber iniciado sesión previamente. En caso de creación, el nombre del autor no debe estar registrado.

4. Implementación

4.1 Tecnologías utilizadas

- Backend: PHP y el framework Laravel
- Frontend: Angular
- Servidor web: Apache
- Base de datos: MariaDB
- Autenticación: Laravel Sanctum + Laravel Socialite (Google OAuth)
- Formato de datos: JSON
- Protocolo: HTTPS

4.2 Documentación de la API

Esta documentación describe la API REST que da soporte al backend de la aplicación web de Préstamo de Libros. La aplicación permite a usuarios registrados añadir libros de su propiedad, ponerlos disponibles para préstamo y solicitar libros de otros usuarios.

4.2.1 URL Base

Todas las rutas de la API tienen el siguiente prefijo:

```
https://api.prestamolibros.com/api/v1
```

4.2.2 Autenticación

La API utiliza autenticación mediante Laravel Sanctum con Personal Access Tokens. Sanctum es la solución oficial de Laravel para autenticar SPAs y APIs. Además, se integra de forma nativa con Laravel Socialite para el inicio de sesión con Google.

Tras iniciar sesión (por email/contraseña o mediante Google OAuth), el servidor devuelve un token de acceso que el cliente debe incluir en la cabecera Authorization de todas las peticiones protegidas:

```
Authorization: Bearer <token>
```

Los tokens son cadenas opacas generadas aleatoriamente y almacenadas de forma hasheada en la tabla `personal_access_tokens`. La autenticación se valida en el servidor comparando el token recibido con los registros almacenados.

Los tokens pueden revocarse en cualquier momento (por ejemplo, al hacer logout), garantizando un cierre de sesión inmediato.

El inicio de sesión mediante Google se implementa usando Laravel Socialite. El cliente obtiene un código de autorización de Google y lo envía a la API, donde el servidor valida la identidad del usuario y emite un token Sanctum equivalente al login tradicional.

Los endpoints de autenticación no requieren token. Gestionan el registro, login y logout de usuarios.

4.2.3 Endpoints

Autenticación

- Registro de usuario

POST /auth/register	
Crea una nueva cuenta de usuario en el sistema.	
Autenticación	No requerida
Body (JSON)	name (string, requerido) — Nombre del usuario lastname (string, requerido) — Apellidos del usuario email (string, requerido) — Email único (usado para login) password (string, requerido) — Contraseña (mínimo 8 caracteres)
Respuesta	201 Usuario creado. Devuelve objeto user y token JWT 400 Faltan campos requeridos 409 El email ya está registrado
Notas	La contraseña se almacena cifrada con <code>bcrypt</code> . El rol por defecto es 'user'.

- Login

POST /auth/login	
Autentica al usuario con email y contraseña. Devuelve un Personal Access Token de Laravel Sanctum	
Autenticación	No requerida
Body (JSON)	email (string, requerido) — Email del usuario password (string, requerido) — Contraseña del usuario
Respuesta	200 Login exitoso. Devuelve token JWT y datos del usuario 400 Campos faltantes 401 Credenciales incorrectas

- Logout

POST /auth/logout	
<i>Invalida el token actual del usuario.</i>	
Autenticación	Bearer Token requerido
Respuesta	204 Logout exitoso 401 Token no válido o ausente

- Login con Google

POST /auth/google	
<i>Autentica al usuario mediante OAuth2 con cuenta de Google.</i>	
Autenticación	No requerida (OAuth2 Google)
Body (JSON)	id_token (string, requerido) — Token de Google obtenido en el cliente
Respuesta	200 Autenticación exitosa. Devuelve token JWT y datos del usuario 401 Token de Google inválido o caducado
Notas	<i>Requiere configurar OAuth2 Client ID en Google Cloud Console y verificar el token en el servidor usando la librería oficial de Google.</i>

Usuarios

- Obtener perfil propio

GET /users/me	
<i>Devuelve los datos del usuario autenticado.</i>	
Autenticación	Bearer Token requerido
Respuesta	200 Devuelve objeto user (sin password) 401 Token no válido

- Actualizar perfil propio

PUT /users/me	
<i>Actualiza los datos del usuario autenticado.</i>	
Autenticación	Bearer Token requerido
Body (JSON)	name (string, opcional) — Nuevo nombre lastname (string, opcional) — Nuevos apellidos email (string, opcional) — Nuevo email password (string, opcional) — Nueva contraseña
Respuesta	200 Datos actualizados correctamente 409 El email ya está en uso

- Listar todos los usuarios (Admin)

GET /admin/users	
<i>Lista todos los usuarios registrados.</i>	
Autenticación	Bearer Token requerido + rol admin
Parámetros URL	page (integer) — Número de página (paginación) per_page (integer) — Resultados por página (defecto 20)
Respuesta	200 Array de objetos user 403 Sin permisos de administrador

- Eliminar usuario (Admin)

DELETE /admin/users/:id	
<i>Elimina un usuario del sistema.</i>	
Autenticación	Bearer Token requerido + rol admin
Parámetros URL	id (integer) — ID del usuario a eliminar
Respuesta	204 Usuario eliminado correctamente 409 El usuario tiene préstamos activos y no puede eliminarse 404 Usuario no encontrado
Notas	<i>No se puede eliminar un usuario si tiene préstamos asociados (activos o históricos). Verificar antes de eliminar.</i>

Libros

- Listar libros (con búsqueda)

GET /books	
<i>Lista todos los libros disponibles. Permite búsqueda por título, autor o género.</i>	
Autenticación	Bearer Token requerido
Parámetros URL	search (string) — Texto a buscar (busca en título, autor y género) genre (string) — Filtrar por nombre de género author (string) — Filtrar por nombre de autor page (integer) — Página de resultados per_page (integer) — Resultados por página
Respuesta	200 Array paginado de libros con datos de autor, género y disponibilidad
Notas	<i>La respuesta incluye cuántas copias hay disponibles de cada libro.</i>

- Obtener un libro

GET /books/:id	
Devuelve los datos de un libro concreto.	
Autenticación	Bearer Token requerido
Parámetros URL	id (integer) — ID del libro
Respuesta	200 Objeto libro con autor, género, propietario y copias 404 Libro no encontrado

- Crear libro

POST /books	
Crea un nuevo libro. El propietario (owner_id) se asigna automáticamente al usuario autenticado.	
Autenticación	Bearer Token requerido
Body (JSON)	title (string, requerido) — Título del libro author_id (integer, requerido) — ID del autor (debe existir en la tabla authors) genre_id (integer, requerido) — ID del género (debe existir en la tabla genres) publication_year (integer, opcional) — Año de publicación (4 dígitos)
Respuesta	201 Libro creado. Devuelve objeto book 400 Faltan campos requeridos 422 Año inválido u otros errores de validación
Notas	El campo owner_id se asigna automáticamente en el servidor. El cliente no puede especificar otro propietario.

- Actualizar libro

PUT /books/:id	
Actualiza los datos de un libro. Solo el propietario puede editarlo.	
Autenticación	Bearer Token requerido
Parámetros URL	id (integer) — ID del libro
Body (JSON)	title (string, opcional) — Nuevo título author_id (integer, opcional) — Nuevo autor genre_id (integer, opcional) — Nuevo género publication_year (integer, opcional) — Nuevo año
Respuesta	200 Libro actualizado 403 El usuario no es el propietario del libro 404 Libro no encontrado

- Eliminar libro

DELETE /books/:id	
<i>Elimina un libro. Solo el propietario puede eliminarlo.</i>	
Autenticación	Bearer Token requerido
Parámetros URL	id (integer) — ID del libro
Respuesta	204 Libro eliminado 403 El usuario no es el propietario 409 El libro tiene préstamos activos y no puede eliminarse

- Importar libros por CSV

POST /books/import	
<i>Importación masiva de libros desde un archivo CSV.</i>	
Autenticación	Bearer Token requerido
Body (JSON)	file (file (multipart), requerido) — Archivo CSV con columnas: title, author, genre, publication_year
Respuesta	200 Importación completada. Devuelve resumen: libros creados, errores por fila 400 Formato de archivo incorrecto o CSV con estructura inválida 422 Errores de validación en los datos del CSV
Notas	<i>El CSV debe tener cabeceras en la primera fila. Se crearán autores y géneros nuevos automáticamente si no existen. Los errores por fila no detienen la importación completa.</i>

- Libros del usuario autenticado

GET /users/me/books	
<i>Devuelve los libros que el usuario autenticado ha dado de alta (es propietario).</i>	
Autenticación	Bearer Token requerido
Respuesta	200 Array de libros con información de copias y préstamos activos

Copias

- Listar copias de un libro

GET /books/:id/copies	
Devuelve todas las copias de un libro con su estado.	
Autenticación	Bearer Token requerido
Parámetros URL	id (integer) — ID del libro
Respuesta	200 Array de copias con estado y código 404 Libro no encontrado

- Añadir copia a un libro

POST /books/:id/copies	
Añade una nueva copia de un libro. Solo el propietario puede hacerlo.	
Autenticación	Bearer Token requerido
Parámetros URL	id (integer) — ID del libro
Body (JSON)	code (string, requerido) — Código único identificador de la copia (ej. ISBN, código propio)
Respuesta	201 Copia creada con estado 'available' 403 El usuario no es el propietario del libro 409 El código ya existe

- Eliminar copia

DELETE /copies/:id	
Elimina una copia. No se puede eliminar si tiene préstamos activos.	
Autenticación	Bearer Token requerido
Parámetros URL	id (integer) — ID de la copia
Respuesta	204 Copia eliminada 403 Sin permisos 409 La copia está prestada actualmente

Préstamos

- Solicitar préstamo

POST /loans	
El usuario autenticado solicita en préstamo una copia de un libro.	
Autenticación	Bearer Token requerido
Body (JSON)	<code>copy_id</code> (integer, requerido) — ID de la copia que se desea tomar prestada
Respuesta	201 Préstamo creado. Devuelve objeto loan con loan_date 400 La copia no está disponible (ya prestada) 409 El usuario ya tiene esta copia en préstamo
Notas	La fecha de préstamo (loan_date) se asigna automáticamente en el servidor con la fecha actual. El estado de la copia cambia a 'borrowed'.

- Devolver libro (cerrar préstamo)

PATCH /loans/:id/return	
Marca un préstamo como devuelto. Solo el usuario que solicitó el préstamo puede devolver el libro.	
Autenticación	Bearer Token requerido
Parámetros URL	<code>id</code> (integer) — ID del préstamo
Respuesta	200 Préstamo cerrado. Se asigna return_date con la fecha actual. Estado de la copia vuelve a 'available' 403 El usuario no es quien tiene el préstamo 404 Préstamo no encontrado 409 El préstamo ya está cerrado (ya fue devuelto)

- Préstamos activos

GET /users/me/loans	
Devuelve los préstamos activos del usuario autenticado (libros que tiene prestados actualmente).	
Autenticación	Bearer Token requerido
Respuesta	200 Array de préstamos activos con datos del libro, copia y días transcurridos desde loan_date
Notas	La respuesta incluye el campo days_elapsed calculado como la diferencia en días entre loan_date y la fecha actual.

- Libros prestados (como propietario)

GET /users/me/lent	
Devuelve los préstamos activos de libros que pertenecen al usuario autenticado.	
Autenticación	Bearer Token requerido
Respuesta	200 Array de préstamos activos de los libros del usuario, con información del prestatario y días transcurridos

- Todos los préstamos (Admin)

GET /admin/loans	
Lista todos los préstamos del sistema (activos e históricos).	
Autenticación	Bearer Token requerido + rol admin
Parámetros URL	active (boolean) — Si es true, devuelve solo préstamos activos user_id (integer) — Filtrar por usuario page (integer) — Paginación
Respuesta	200 Array paginado de todos los préstamos 403 Sin permisos de administrador

Géneros

- Listar géneros

GET /genres	
Devuelve todos los géneros disponibles.	
Autenticación	Bearer Token requerido
Respuesta	200 Array de géneros [{id, name}]

- Crear género

POST /genres	
Crea un nuevo género.	
Autenticación	Bearer Token requerido + rol admin
Body (JSON)	name (string, requerido) — Nombre del género (único)
Respuesta	201 Género creado 409 El género ya existe

Autores

- Listar autores

GET /authors	
Devuelve todos los autores disponibles. Permite búsqueda por nombre.	
Autenticación	Bearer Token requerido
Parámetros URL	search (string) — Buscar por nombre de autor
Respuesta	200 Array de autores [{id, name}]

- Crear autor

POST /authors	
Crea un nuevo autor.	
Autenticación	Bearer Token requerido
Body (JSON)	name (string, requerido) — Nombre del autor (único)
Respuesta	201 Autor creado 409 El autor ya existe

4.2.4 Códigos de Respuesta HTTP

La API utiliza los siguientes códigos HTTP estándar:

Código HTTP	Descripción
200 OK	Solicitud exitosa
201 Created	Recurso creado correctamente
204 No Content	Eliminación correcta, sin cuerpo de respuesta
400 Bad Request	Parámetros incorrectos o falta de campos requeridos
401 Unauthorized	Token no proporcionado o inválido
403 Forbidden	Sin permisos para realizar la acción
404 Not Found	Recurso no encontrado
409 Conflict	Conflicto (ej. email ya registrado, libro ya prestado)
422 Unprocessable Entity	Datos con formato inválido (ej. año no numérico)
500 Internal Server Error	Error interno del servidor

Formato de respuesta de error:

Cuando ocurre un error, la API devuelve siempre el mismo formato JSON:

```
{
  "error": true,
  "code": 404,
  "message": "Recurso no encontrado",
  "details": "No existe ningún libro con el ID proporcionado"
}
```

4.2.5 Ejemplos de uso

- Registro y login

Registro

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "name": "Juan",
  "lastname": "García López",
  "email": "juan@ejemplo.com",
  "password": "contraseña123"
}
```

Respuesta exitosa

```
HTTP/1.1 201 Created
{
  "user": { "id": 5, "name": "Juan", "email": "juan@ejemplo.com", "role": "user" },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

- Crear un libro

```
POST /api/v1/books
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "title": "El Quijote",
  "author_id": 3,
  "genre_id": 1,
  "publication_year": 1605
}
```

- Buscar libros

```
GET /api/v1/books?search=quijote&genre=Novela&page=1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

- Solicitar préstamo

```
POST /api/v1/loans
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "copy_id": 12
}
```

Respuesta

```
HTTP/1.1 201 Created
{
  "id": 45,
  "user_id": 5,
  "copy_id": 12,
  "loan_date": "2026-02-24",
  "return_date": null,
  "days_elapsed": 0
}
```

- Importación CSV

```
POST /api/v1/books/import
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: multipart/form-data

[archivo: libros.csv]
```

Formato del CSV

```
title,author,genre,publication_year
El Quijote,Miguel de Cervantes,Novela,1605
Cien años de soledad,Gabriel García Márquez,Novela,1967
1984,George Orwell,Ciencia Ficción,1949
```

4.2.6 Resumen de Endpoints

Método	Ruta	Descripción	Auth
POST	/auth/register	Registro de usuario	No
POST	/auth/login	Login	No
POST	/auth/logout	Logout	Sí
POST	/auth/google	Login con Google ★	No
GET	/users/me	Perfil propio	Sí
PUT	/users/me	Actualizar perfil	Sí
GET	/admin/users	Listar usuarios (admin)	Admin
DELETE	/admin/users/:id	Eliminar usuario (admin)	Admin
GET	/books	Listar/buscar libros	Sí
GET	/books/:id	Obtener libro	Sí
POST	/books	Crear libro	Sí
PUT	/books/:id	Actualizar libro	Sí
DELETE	/books/:id	Eliminar libro	Sí
POST	/books/import	Importar CSV ★	Sí
GET	/users/me/books	Mis libros	Sí
GET	/books/:id/copies	Copias de un libro	Sí
POST	/books/:id/copies	Añadir copia	Sí
DELETE	/copies/:id	Eliminar copia	Sí
POST	/loans	Solicitar préstamo	Sí
PATCH	/loans/:id/return	Devolver libro	Sí
GET	/users/me/loans	Mis préstamos activos	Sí
GET	/users/me/lent	Mis libros prestados	Sí
GET	/admin/loans	Todos los préstamos (admin)	Admin
GET	/genres	Listar géneros	Sí
POST	/genres	Crear género	Admin
GET	/authors	Listar autores	Sí
POST	/authors	Crear autor	Sí

5. Bibliografía y recursos usados

6. Anexos
